

Teach Module 24 README

Module 24: SOAP Web Services With Spring WS

Module 24 covers SOAP Web Services with Spring WS, especially SOAP endpoints, request mapping, WSDL/XSD generation, SOAP faults, and basic security concepts.

These notes use the course document only to identify the module topic. The teaching content below is written independently and does not reuse course material.

Core Concept

SOAP is a protocol for exchanging structured XML messages between systems. It is common in banking, insurance, government, healthcare, legacy enterprise systems, and business-to-business integrations.

Unlike REST, where you usually expose resources like:

```
GET /accounts/123
POST /payments
```

SOAP exposes operations through XML messages, often described by a formal contract called a WSDL.

Think of SOAP like this:

```
Client sends XML request
      |
SOAP endpoint receives it
      |
Spring WS maps XML to Java
      |
Business logic runs
      |
Java response becomes XML
      |
SOAP response returns to client
```

SOAP Message Structure

A SOAP message is XML wrapped inside an envelope.

```
<soapenv:Envelope>
  <soapenv:Header>
    <!-- metadata, security, tracing -->
  </soapenv:Header>

  <soapenv:Body>
    <GetCustomerRequest>
      <customerId>101</customerId>
    </GetCustomerRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

```
</soapenv:Body>
</soapenv:Envelope>
```

Important parts:

- **Envelope** : the outer wrapper.
- **Header** : optional metadata such as authentication, transaction IDs, or routing information.
- **Body** : the actual request or response payload.
- **Fault** : standard error response format when something goes wrong.

Spring WS Endpoint

In Spring Web Services, a SOAP endpoint is a Java class that receives SOAP XML requests.

```
@Endpoint
public class CustomerEndpoint {

    private static final String NAMESPACE_URI = "http://example.com/customers";

    private final CustomerService customerService;

    public CustomerEndpoint(CustomerService customerService) {
        this.customerService = customerService;
    }

    @PayloadRoot(namespace = NAMESPACE_URI, localPart = "GetCustomerRequest")
    @ResponsePayload
    public GetCustomerResponse getCustomer(@RequestPayload GetCustomerRequest request) {
        Customer customer = customerService.findById(request.getCustomerId());

        GetCustomerResponse response = new GetCustomerResponse();
        response.setCustomer(customer);

        return response;
    }
}
```

Key annotations:

- **@Endpoint** : marks the class as a SOAP web service endpoint.
- **@PayloadRoot** : tells Spring which XML request this method handles.
- **@RequestPayload** : maps incoming XML into a Java object.
- **@ResponsePayload** : maps the Java return object back into SOAP XML.

Request Mapping in SOAP

In REST, mapping usually depends on HTTP method and URL:

```
@GetMapping("/customers/{id}")
```

In SOAP, mapping depends on the XML payload:

```
@PayloadRoot(namespace = "...", localPart = "GetCustomerRequest")
```

Spring asks whether the incoming SOAP body contains a `GetCustomerRequest` element in the expected namespace. If yes, it calls the matching endpoint method.

XSD: The Data Contract

SOAP services commonly use an XSD file to define request and response shapes.

```
<xs:element name="GetCustomerRequest">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="customerId" type="xs:int"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

This means the request must contain a `customerId`, and it must be an integer.

The XSD is the schema rulebook for the XML.

WSDL: The Service Contract

WSDL describes the full SOAP service:

- What operations exist.
- What request XML each operation expects.
- What response XML it returns.
- Where the service is located.
- What protocol and binding it uses.

A SOAP client can use the WSDL to generate Java client code automatically. This is one reason SOAP is still popular in enterprise environments: the contract is strict and machine-readable.

SOAP Faults

SOAP has a standard way to return errors called a Fault.

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid customer ID</faultstring>
</soap:Fault>
```

Common fault categories:

- `Client` : the request was invalid.
- `Server` : something failed on the server.

In Spring WS, Java exceptions can be mapped into SOAP faults using exception resolvers or endpoint exception handling.

WS-Security Basics

SOAP often uses WS-Security for enterprise-grade security.

It can support:

- Username/password tokens.
- Digital signatures.
- XML encryption.
- Timestamps.
- Message integrity validation.

REST APIs usually secure the transport with HTTPS and tokens like JWT. SOAP can secure the actual XML message itself, which matters when messages pass through multiple systems.

REST vs SOAP

Feature	REST	SOAP
Format	Usually JSON	XML
Contract	OpenAPI optional	WSDL common
Mapping	URL + HTTP method	XML payload
Error format	Flexible	SOAP Fault
Common use	Web/mobile APIs	Enterprise integrations
Security	HTTPS, OAuth, JWT	WS-Security, signatures, encryption

Mental Model

A SOAP service is not mainly an endpoint URL.

It is better understood as:

```
A formal XML contract + operations + schemas + strict request/response rules
```

Spring WS helps Java applications implement that contract cleanly.

Mini Practice

Suppose the SOAP body contains:

```
<GetOrderRequest>
  <orderId>5001</orderId>
</GetOrderRequest>
```

The matching Spring WS method would likely look like:

```

@PayloadRoot(namespace = "http://example.com/orders", localPart = "GetOrderRequest")
@ResponsePayload
public GetOrderResponse getOrder(@RequestPayload GetOrderRequest request) {
    // lookup order
}

```

The key is that `localPart = "GetOrderRequest"` matches the XML element in the SOAP body.

Practice Exercises

Exercise 1: Create a Simple SOAP Endpoint

Build a Spring Boot and Spring WS service called `CustomerService`.

Create a SOAP operation:

```

GetCustomerRequest(customerId)
GetCustomerResponse(customerId, firstName, lastName, email)

```

Practice goals:

- Create an XSD schema.
- Generate JAXB classes from the schema.
- Create an `@Endpoint` class.
- Use `@PayloadRoot`, `@RequestPayload`, and `@ResponsePayload`.
- Test the request using SoapUI, Postman, or curl.

Exercise 2: Add Another Operation

Extend the same SOAP service with:

```

CreateCustomerRequest(firstName, lastName, email)
CreateCustomerResponse(customerId, status)

```

Practice goals:

- Add a second request/response pair to the XSD.
- Add another endpoint method.
- Return a generated customer ID.
- Validate that both operations work from the same SOAP service.

Exercise 3: Generate and Inspect the WSDL

Expose the WSDL at a URL such as:

```

http://localhost:8080/ws/customers.wsdl

```

Practice goals:

- Configure Spring WS `DefaultWsd11Definition`.
- Connect the WSDL to your XSD.
- Open the WSDL in the browser.

- Identify the service name, port, binding, operation, request, and response.

Exercise 4: Validate XML Input

Add validation rules to the XSD.

Examples:

```
customerId must be required
email must be required
firstName minimum length: 2
lastName minimum length: 2
```

Practice goals:

- Use XSD constraints.
- Send invalid SOAP requests.
- Observe how Spring WS rejects invalid payloads.
- Compare valid and invalid request XML.

Exercise 5: Implement SOAP Fault Handling

Create custom errors:

```
CustomerNotFoundException
InvalidCustomerRequestException
```

Practice goals:

- Throw an exception when `customerId` does not exist.
- Convert the exception into a SOAP Fault.
- Return meaningful fault messages.
- Understand the difference between client faults and server faults.

Exercise 6: Add an In-Memory Repository

Use a simple Java `Map<Integer, Customer>` as a fake database.

Practice goals:

- Separate endpoint logic from business logic.
- Create a `CustomerRepository`.
- Create a `CustomerService`.
- Keep the SOAP endpoint thin.

Suggested structure:

```
CustomerEndpoint
CustomerService
CustomerRepository
Customer
```

Exercise 7: Test with Raw SOAP XML

Send a SOAP request manually using curl or Postman.

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:cus="http://example.com/customers">
    <soapenv:Header/>
    <soapenv:Body>
        <cus:GetCustomerRequest>
            <cus:customerId>1</cus:customerId>
        </cus:GetCustomerRequest>
    </soapenv:Body>
</soapenv:Envelope>
```

Practice goals:

- Understand the SOAP envelope.
- Understand namespaces.
- Understand the body payload.
- Debug mapping errors.

Exercise 8: Break the Namespace on Purpose

Change the namespace in the request XML to something incorrect.

Practice goals:

- See why `@PayloadRoot(namespace = "...")` matters.
- Learn how SOAP request mapping fails.
- Practice reading Spring WS error messages.

Exercise 9: Add WS-Security Username Token

Add basic SOAP security using a username/password token.

Practice goals:

- Understand SOAP headers.
- Add security-related configuration.
- Reject requests without credentials.
- Accept requests with valid credentials.

This is more advanced, so do it after the basic endpoint works.

Exercise 10: Build a SOAP Client

Create a separate Java client that calls your SOAP service.

Practice goals:

- Use `WebServiceTemplate`.
- Send a request object.
- Receive a response object.
- Handle SOAP faults on the client side.

Best Practice Sequence

1. Simple `GetCustomer` endpoint.

2. XSD schema.
3. WSDL exposure.
4. Manual SOAP testing.
5. Add `CreateCustomer`.
6. Add validation.
7. Add SOAP faults.
8. Add service/repository layers.
9. Add SOAP client.
10. Add basic WS-Security.

The most important exercises are 1, 3, 5, and 7. If you can build an endpoint, expose WSDL, handle faults, and test with raw XML, you understand the core of Module 24.

Lab: Build a SOAP Web Service With Spring WS

Goal

Create a Spring Boot SOAP service for managing customers.

You will build:

```
GetCustomerRequest
GetCustomerResponse
CustomerEndpoint
CustomerService
CustomerRepository
customers.xsd
customers.wsdl
```

Lab Scenario

Your company has an older enterprise system that expects SOAP XML messages. You need to expose a SOAP service that allows clients to retrieve customer details by customer ID.

Part 1: Create the Project

Create a Spring Boot project with these dependencies:

```
Spring Web Services
Spring Web
Spring Boot DevTools
```

If using Maven, include dependencies similar to:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web-services</artifactId>
</dependency>

<dependency>
  <groupId>wsdl4j</groupId>
```



```
<artifactId>wsdl4j</artifactId>
</dependency>
```

Part 2: Create the XSD

Create:

```
src/main/resources/customers.xsd
```

Add a schema with:

```
GetCustomerRequest
GetCustomerResponse
Customer
```

Suggested fields:

```
customerId
firstName
lastName
email
accountType
```

Example:

```
<xs:element name="GetCustomerRequest">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="customerId" type="xs:int"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Task: Define the response so it returns customer details.

Part 3: Generate Java Classes

Configure Maven to generate Java classes from the XSD.

Expected generated classes:

```
GetCustomerRequest
GetCustomerResponse
Customer
ObjectFactory
```

Then run:

```
mvn clean compile
```

Part 4: Configure Spring WS

Create a configuration class:

```
@EnableWs
@Configuration
public class WebServiceConfig extends WsConfigurerAdapter {
}
```

Inside it, configure:

```
MessageDispatcherServlet
DefaultWsd11Definition
XsdSchema
```

The WSDL should be available at:

```
http://localhost:8080/ws/customers.wsdl
```

Part 5: Create Repository

Create an in-memory repository:

```
@Repository
public class CustomerRepository {

    private final Map<Integer, Customer> customers = new HashMap<>();

    public CustomerRepository() {
        Customer customer = new Customer();
        customer.setCustomerId(1);
        customer.setFirstName("Ava");
        customer.setLastName("Patel");
        customer.setEmail("ava.patel@example.com");
        customer.setAccountType("Premium");

        customers.put(1, customer);
    }

    public Customer findById(int customerId) {
        return customers.get(customerId);
    }
}
```

Part 6: Create SOAP Endpoint

Create:

```
@Endpoint
public class CustomerEndpoint {
}
```

Add a method that handles:

```
GetCustomerRequest
```

Use:

```
@PayloadRoot
@RequestPayload
@ResponsePayload
```

Expected behavior:

```
Input: customerId = 1
Output: customer details
```

Part 7: Test the WSDL

Run the application:

```
mvn spring-boot:run
```

Open:

```
http://localhost:8080/ws/customers.wsdl
```

Confirm that you can see the generated WSDL.

Part 8: Send SOAP Request

Use Postman, SoapUI, or curl.

Sample SOAP request:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:cus="http://example.com/customers">
  <soapenv:Header/>
  <soapenv:Body>
    <cus:GetCustomerRequest>
      <cus:customerId>1</cus:customerId>
    </cus:GetCustomerRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

Expected response:

```
<GetCustomerResponse>
  <customer>
    <customerId>1</customerId>
    <firstName>Ava</firstName>
    <lastName>Patel</lastName>
    <email>ava.patel@example.com</email>
    <accountType>Premium</accountType>
  </customer>
</GetCustomerResponse>
```

Part 9: Add Fault Handling

If the customer does not exist, return a SOAP fault.

Test with:

```
<cus:customerId>999</cus:customerId>
```

Expected result:

Customer not found

Part 10: Challenge

Add a second operation:

```
GetAllCustomersRequest
GetAllCustomersResponse
```

The response should return multiple customers.

Lab Completion Checklist

You are done when:

- Application starts successfully.
- WSDL is visible in browser.
- SOAP request maps to endpoint method.
- Valid customer ID returns customer data.
- Invalid customer ID returns SOAP fault.
- XSD defines request and response structure.
- Endpoint uses `@PayloadRoot` correctly.

Reflection Questions

1. What does `@PayloadRoot` match against?
2. Why does SOAP need namespaces?
3. What is the difference between XSD and WSDL?
4. How is SOAP fault handling different from REST error handling?
5. Why might an enterprise system prefer SOAP over REST?